

Car Rental / Ride-Hailing System Design

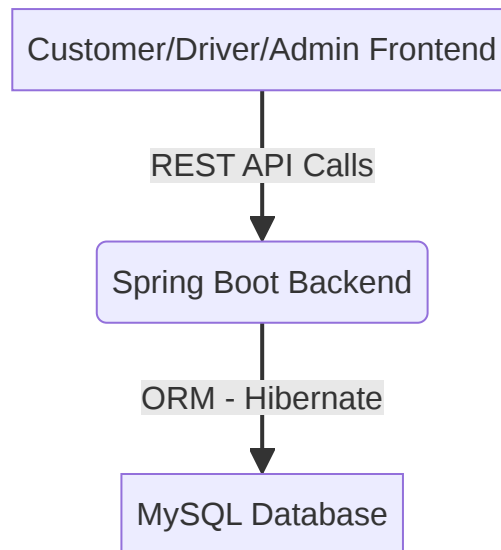
1. Introduction

This document outlines the comprehensive system design for a Car Rental / Ride-Hailing System, similar to popular platforms like Uber and Ola. The goal is to provide a detailed architectural overview, module breakdown, database design, API specifications, frontend considerations, and a step-by-step explanation of the core ride booking flow. This guide is tailored for beginner developers, ensuring clarity and simplicity in all explanations.

2. High-Level System Architecture

The system will follow a classic three-tier architecture, separating concerns into the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database). This separation enhances scalability, maintainability, and flexibility.

- **Frontend (Presentation Layer):** Built with React, Bootstrap, and CSS, this layer is responsible for the user interface and user experience. It interacts with the backend via RESTful APIs to send user requests and display information.
- **Backend (Application Layer):** Developed using Spring Boot, this layer acts as the brain of the system. It handles business logic, processes requests from the frontend, interacts with the database, and exposes RESTful APIs for the frontend to consume.
- **Database (Data Layer):** MySQL will be used as the relational database, managed by Hibernate (ORM). It stores all persistent data, including user profiles, vehicle information, ride details, payment transactions, and more.



3. Module Breakdown

This section details the various modules required to build the Car Rental / Ride-Hailing System. Each module's purpose, functionalities, interacting users, database tables, and API endpoints are described.

3.1. Authentication & Authorization Module

- **Purpose:** To securely manage user registration, login, and access control within the system.
- **Functionalities:** User registration (customer, driver, admin), login (username/password), password reset, token generation (JWT), role-based access control.
- **User Interaction:** Admin, Driver, Customer

Database Tables:

- **users table:**
 - `user_id` (Primary Key, UUID)
 - `username` (Unique, String)
 - `password_hash` (String)
 - `email` (Unique, String)
 - `phone_number` (Unique, String)

- `role` (Enum: 'CUSTOMER', 'DRIVER', 'ADMIN')
- `created_at` (Timestamp)
- `updated_at` (Timestamp)

Relationships:

- `users` table is central and linked to `customers`, `drivers`, and `admins` tables (if separate profiles are maintained).

REST APIs:

- `POST /api/auth/register` (Public): Register a new user.
- `POST /api/auth/login` (Public): Authenticate user and return JWT token.
- `POST /api/auth/refresh-token` (Public): Refresh an expired JWT token.
- `POST /api/auth/forgot-password` (Public): Initiate password reset.
- `POST /api/auth/reset-password` (Public): Complete password reset.

3.2. User Management Module

- **Purpose:** To manage the profiles and basic information of all users (customers, drivers, admins).
- **Functionalities:** View profile, edit profile, update contact information, manage addresses (for customers), view associated roles.
- **User Interaction:** Admin, Driver, Customer

Database Tables:

- **customers table:**
 - `customer_id` (Primary Key, UUID)
 - `user_id` (Foreign Key to `users.user_id`)
 - `first_name` (String)
 - `last_name` (String)
 - `profile_picture_url` (String, Optional)

- `default_address` (String, Optional)
- `created_at` (Timestamp)
- `updated_at` (Timestamp)
- **drivers table:**
 - `driver_id` (Primary Key, UUID)
 - `user_id` (Foreign Key to `users.user_id`)
 - `first_name` (String)
 - `last_name` (String)
 - `license_number` (Unique, String)
 - `vehicle_id` (Foreign Key to `vehicles.vehicle_id`, Optional)
 - `status` (Enum: 'AVAILABLE', 'ON_RIDE', 'OFFLINE', 'PENDING_APPROVAL')
 - `current_location` (Point/GeoJSON, for real-time tracking)
 - `rating` (Decimal)
 - `created_at` (Timestamp)
 - `updated_at` (Timestamp)
- **admins table:**
 - `admin_id` (Primary Key, UUID)
 - `user_id` (Foreign Key to `users.user_id`)
 - `first_name` (String)
 - `last_name` (String)
 - `created_at` (Timestamp)
 - `updated_at` (Timestamp)

Relationships:

- `customers.user_id` -> `users.user_id`
- `drivers.user_id` -> `users.user_id`
- `admins.user_id` -> `users.user_id`

REST APIs:

- `GET /api/users/{id}` (Authenticated): Get user profile by ID.
- `PUT /api/users/{id}` (Authenticated): Update user profile.
- `GET /api/customers/{id}` (Authenticated, Admin/Customer): Get customer profile.
- `PUT /api/customers/{id}` (Authenticated, Customer): Update customer profile.
- `GET /api/drivers/{id}` (Authenticated, Admin/Driver): Get driver profile.
- `PUT /api/drivers/{id}` (Authenticated, Driver): Update driver profile.
- `GET /api/admins/{id}` (Authenticated, Admin): Get admin profile.
- `PUT /api/admins/{id}` (Authenticated, Admin): Update admin profile.

3.3. Driver Management Module

- **Purpose:** To manage driver-specific information, including vehicle assignments, status, and approval processes.
- **Functionalities:** Driver registration approval, driver status management (online/offline, available/on-ride), vehicle assignment, document verification.
- **User Interaction:** Admin, Driver

Database Tables:

- `drivers` (as defined in User Management)
- **driver_documents table:**
 - `document_id` (Primary Key, UUID)
 - `driver_id` (Foreign Key to `drivers.driver_id`)
 - `document_type` (Enum: 'LICENSE', 'REGISTRATION', 'INSURANCE', 'BACKGROUND_CHECK')
 - `document_url` (String)
 - `status` (Enum: 'PENDING', 'APPROVED', 'REJECTED')
 - `uploaded_at` (Timestamp)
 - `approved_by` (Foreign Key to `admins.admin_id`, Optional)

Relationships:

- `driver_documents.driver_id` -> `drivers.driver_id`

REST APIs:

- `GET /api/drivers` (Authenticated, Admin): Get all drivers.
- `PUT /api/drivers/{id}/status` (Authenticated, Admin/Driver): Update driver status.
- `POST /api/drivers/{id}/documents` (Authenticated, Driver): Upload driver documents.
- `PUT /api/drivers/documents/{documentId}/status` (Authenticated, Admin): Update document status (approve/reject).
- `GET /api/drivers/pending-approval` (Authenticated, Admin): Get drivers awaiting approval.

3.4. Vehicle Management Module

- **Purpose:** To manage all vehicle-related information, including details, availability, and assignment to drivers.
- **Functionalities:** Add/edit vehicle details, assign/unassign vehicles to drivers, track vehicle status (active, inactive, maintenance).
- **User Interaction:** Admin, Driver

Database Tables:

- **vehicles table:**
 - `vehicle_id` (Primary Key, UUID)
 - `driver_id` (Foreign Key to `drivers.driver_id`, Optional, Unique)
 - `make` (String)
 - `model` (String)
 - `year` (Integer)
 - `license_plate` (Unique, String)
 - `color` (String)

- `capacity` (Integer)
- `vehicle_type` (Enum: 'SEDAN', 'SUV', 'BIKE', 'LUXURY')
- `status` (Enum: 'ACTIVE', 'IN_MAINTENANCE', 'INACTIVE')
- `created_at` (Timestamp)
- `updated_at` (Timestamp)

Relationships:

- `vehicles.driver_id -> drivers.driver_id` (One-to-one or One-to-many depending on business logic, assuming one-to-one for simplicity initially).

REST APIs:

- `POST /api/vehicles` (Authenticated, Admin): Add a new vehicle.
- `GET /api/vehicles/{id}` (Authenticated, Admin/Driver): Get vehicle details by ID.
- `PUT /api/vehicles/{id}` (Authenticated, Admin): Update vehicle details.
- `DELETE /api/vehicles/{id}` (Authenticated, Admin): Delete a vehicle.
- `PUT /api/vehicles/{id}/assign-driver` (Authenticated, Admin): Assign vehicle to a driver.
- `PUT /api/vehicles/{id}/unassign-driver` (Authenticated, Admin): Unassign vehicle from a driver.

3.5. Ride Booking Module

- **Purpose:** To allow customers to request rides and manage the lifecycle of a ride request.
- **Functionalities:** Request a ride (pickup, drop-off, vehicle type), view estimated fare, cancel ride request, view ride status.
- **User Interaction:** Customer

Database Tables:

- **rides table:**
 - `ride_id` (Primary Key, UUID)

- `customer_id` (Foreign Key to `customers.customer_id`)
- `driver_id` (Foreign Key to `drivers.driver_id`, Optional)
- `pickup_location` (Point/GeoJSON)
- `dropoff_location` (Point/GeoJSON)
- `pickup_address` (String)
- `dropoff_address` (String)
- `vehicle_type_requested` (Enum: 'SEDAN', 'SUV', 'BIKE', 'LUXURY')
- `status` (Enum: 'PENDING', 'SEARCHING_DRIVER', 'ACCEPTED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED')
- `fare_estimate` (Decimal)
- `actual_fare` (Decimal, Optional)
- `start_time` (Timestamp, Optional)
- `end_time` (Timestamp, Optional)
- `created_at` (Timestamp)
- `updated_at` (Timestamp)

Relationships:

- `rides.customer_id` -> `customers.customer_id`
- `rides.driver_id` -> `drivers.driver_id`

REST APIs:

- `POST /api/rides/request` (Authenticated, Customer): Request a new ride.
- `GET /api/rides/{id}` (Authenticated, Customer/Driver/Admin): Get ride details by ID.
- `PUT /api/rides/{id}/cancel` (Authenticated, Customer/Driver): Cancel a ride.
- `GET /api/rides/{id}/status` (Authenticated, Customer/Driver): Get current ride status.
- `GET /api/rides/estimate-fare` (Authenticated, Customer): Get estimated fare for a ride.

3.6. Ride Matching Module

- **Purpose:** To efficiently match ride requests from customers with available drivers based on proximity and other criteria.
- **Functionalities:** Find nearby drivers, send ride requests to drivers, driver acceptance/rejection of rides, dynamic re-matching.
- **User Interaction:** Driver (implicitly by receiving requests), Backend (automated process)

Database Tables:

- `rides` (as defined in Ride Booking)
- `drivers` (as defined in User Management)

Relationships:

- Relies on `rides` and `drivers` tables.

REST APIs:

- `GET /api/drivers/nearby` (Authenticated, Backend/Internal): Find nearby available drivers for a given location.
- `POST /api/rides/{rideId}/assign-driver` (Authenticated, Backend/Internal): Assign a driver to a ride.
- `POST /api/drivers/{driverId}/accept-ride` (Authenticated, Driver): Driver accepts a ride request.
- `POST /api/drivers/{driverId}/reject-ride` (Authenticated, Driver): Driver rejects a ride request.

3.7. Location & Map Tracking Module

- **Purpose:** To provide real-time location tracking for drivers and rides, and display this information on maps.
- **Functionalities:** Real-time driver location updates, ride progress tracking, display pickup/drop-off points, route visualization.
- **User Interaction:** Customer, Driver, Admin

Database Tables:

- `drivers` (for `current_location`)
- `rides` (for `pickup_location`, `dropoff_location`)
- **`ride_locations` table (for historical tracking/debugging):**
 - `location_id` (Primary Key, UUID)
 - `ride_id` (Foreign Key to `rides.ride_id`)
 - `latitude` (Decimal)
 - `longitude` (Decimal)
 - `timestamp` (Timestamp)

Relationships:

- `ride_locations.ride_id` -> `rides.ride_id`

REST APIs:

- `PUT /api/drivers/{id}/location` (Authenticated, Driver): Update driver's current location.
- `GET /api/rides/{id}/track` (Authenticated, Customer/Admin): Get real-time ride location updates.
- `GET /api/drivers/{id}/current-location` (Authenticated, Customer/Admin): Get a specific driver's current location.

3.8. Payment Module

- **Purpose:** To handle all payment-related operations, including fare calculation, payment processing, and transaction history.
- **Functionalities:** Process payments (credit card, digital wallets), manage payment methods, view transaction history, handle refunds.
- **User Interaction:** Customer, Admin

Database Tables:

- **`payments` table:**

- `payment_id` (Primary Key, UUID)
- `ride_id` (Foreign Key to `rides.ride_id`)
- `customer_id` (Foreign Key to `customers.customer_id`)
- `amount` (Decimal)
- `currency` (String, e.g., 'USD')
- `payment_method` (Enum: 'CREDIT_CARD', 'DEBIT_CARD', 'PAYPAL', 'WALLET')
- `transaction_id` (String, from payment gateway)
- `status` (Enum: 'PENDING', 'COMPLETED', 'FAILED', 'REFUNDED')
- `payment_date` (Timestamp)
- **payment_methods table:**
 - `method_id` (Primary Key, UUID)
 - `customer_id` (Foreign Key to `customers.customer_id`)
 - `card_type` (String, e.g., 'VISA', 'MASTERCARD')
 - `last_four_digits` (String)
 - `token` (String, encrypted token from payment gateway)
 - `is_default` (Boolean)
 - `created_at` (Timestamp)
 - `updated_at` (Timestamp)

Relationships:

- `payments.ride_id -> rides.ride_id`
- `payments.customer_id -> customers.customer_id`
- `payment_methods.customer_id -> customers.customer_id`

REST APIs:

- `POST /api/payments/process` (Authenticated, Customer): Process payment for a ride.

- `GET /api/payments/history` (Authenticated, Customer): Get customer's payment history.
- `POST /api/payment-methods` (Authenticated, Customer): Add a new payment method.
- `GET /api/payment-methods` (Authenticated, Customer): Get customer's saved payment methods.
- `DELETE /api/payment-methods/{id}` (Authenticated, Customer): Remove a payment method.
- `POST /api/payments/{id}/refund` (Authenticated, Admin): Initiate a refund for a payment.

3.9. Rating & Review Module

- **Purpose:** To allow users to rate and review rides and drivers, fostering accountability and quality service.
- **Functionalities:** Submit ratings and reviews for drivers/rides, view average ratings, view individual reviews.
- **User Interaction:** Customer, Driver, Admin

Database Tables:

- **ratings table:**
 - `rating_id` (Primary Key, UUID)
 - `ride_id` (Foreign Key to `rides.ride_id`)
 - `customer_id` (Foreign Key to `customers.customer_id`)
 - `driver_id` (Foreign Key to `drivers.driver_id`)
 - `rating_value` (Integer, 1-5)
 - `comment` (Text, Optional)
 - `created_at` (Timestamp)

Relationships:

- `ratings.ride_id -> rides.ride_id`
- `ratings.customer_id -> customers.customer_id`

- `ratings.driver_id -> drivers.driver_id`

REST APIs:

- `POST /api/ratings` (Authenticated, Customer): Submit a ride/driver rating.
- `GET /api/drivers/{id}/ratings` (Authenticated, Customer/Driver/Admin): Get ratings for a specific driver.
- `GET /api/rides/{id}/rating` (Authenticated, Customer/Driver/Admin): Get rating for a specific ride.

3.10. Notification Module

- **Purpose:** To send real-time updates and alerts to users regarding ride status, promotions, and other important information.
- **Functionalities:** Send push notifications, SMS, email alerts for ride status changes, promotions, system announcements.
- **User Interaction:** Customer, Driver, Admin (for sending announcements), Backend (automated)

Database Tables:

- **notifications table:**
 - `notification_id` (Primary Key, UUID)
 - `user_id` (Foreign Key to `users.user_id`)
 - `type` (Enum: 'RIDE_STATUS', 'PROMOTION', 'ANNOUNCEMENT', 'PAYMENT')
 - `message` (Text)
 - `is_read` (Boolean, Default: false)
 - `created_at` (Timestamp)
 - `delivery_status` (Enum: 'SENT', 'FAILED', 'DELIVERED', Optional)

Relationships:

- `notifications.user_id -> users.user_id`

REST APIs:

- `POST /api/notifications/send` (Authenticated, Admin/Backend): Send a notification.
- `GET /api/users/{id}/notifications` (Authenticated, Customer/Driver): Get user's notifications.
- `PUT /api/notifications/{id}/read` (Authenticated, Customer/Driver): Mark notification as read.

3.11. Admin Dashboard Module

- **Purpose:** To provide administrators with tools to monitor, manage, and configure the entire system.
- **Functionalities:** User management (approve drivers, block users), vehicle management, ride monitoring, payment reconciliation, reporting, system configuration.
- **User Interaction:** Admin

Database Tables:

- Relies on all other tables for data.

Relationships:

- Interacts with almost all other tables.

REST APIs:

- `GET /api/admin/dashboard-stats` (Authenticated, Admin): Get overall system statistics.
- `GET /api/admin/users` (Authenticated, Admin): Get all users.
- `PUT /api/admin/users/{id}/block` (Authenticated, Admin): Block a user.
- `GET /api/admin/rides` (Authenticated, Admin): Get all rides.
- `GET /api/admin/payments` (Authenticated, Admin): Get all payments.
- `GET /api/admin/reports/daily-rides` (Authenticated, Admin): Generate daily ride report.

3.12. Trip History Module

- **Purpose:** To allow users to view their past ride details and associated information.
- **Functionalities:** View past rides, ride details (fare, duration, driver, vehicle), ratings given/received.
- **User Interaction:** Customer, Driver

Database Tables:

- `rides` (as defined in Ride Booking)
- `ratings` (as defined in Rating & Review)

Relationships:

- `rides` and `ratings` tables are directly used.

REST APIs:

- `GET /api/customers/{id}/ride-history` (Authenticated, Customer): Get customer's ride history.
- `GET /api/drivers/{id}/ride-history` (Authenticated, Driver): Get driver's ride history.

3.13. Pricing & Fare Calculation Module

- **Purpose:** To dynamically calculate ride fares based on various factors like distance, time, vehicle type, and surge pricing.
- **Functionalities:** Calculate base fare, distance-based fare, time-based fare, apply surge pricing, apply discounts/promotions.
- **User Interaction:** Backend (automated), Admin (for configuring pricing rules)

Database Tables:

- **fare_settings table:**
 - `setting_id` (Primary Key, UUID)

- `vehicle_type` (Enum: 'SEDAN', 'SUV', 'BIKE', 'LUXURY')
- `base_fare` (Decimal)
- `cost_per_km` (Decimal)
- `cost_per_minute` (Decimal)
- `min_fare` (Decimal)
- `created_at` (Timestamp)
- `updated_at` (Timestamp)
- **surge_pricing table:**
 - `surge_id` (Primary Key, UUID)
 - `area` (Point/GeoJSON or Polygon/GeoJSON)
 - `multiplier` (Decimal)
 - `start_time` (Timestamp)
 - `end_time` (Timestamp)
 - `is_active` (Boolean)
 - `created_at` (Timestamp)
 - `updated_at` (Timestamp)

Relationships:

- `rides` table will use `fare_settings` and `surge_pricing` for calculation.

REST APIs:

- `GET /api/fare/estimate` (Public/Authenticated): Get estimated fare for a ride (uses pickup, drop-off, vehicle type).
- `POST /api/admin/fare-settings` (Authenticated, Admin): Configure fare settings.
- `PUT /api/admin/fare-settings/{id}` (Authenticated, Admin): Update fare settings.
- `POST /api/admin/surge-pricing` (Authenticated, Admin): Create surge pricing rule.

- `PUT /api/admin/surge-pricing/{id}` (Authenticated, Admin): Update surge pricing rule.

4. Frontend Components/Pages (React)

Here are the suggested React components/pages for the frontend application:

- **Login/Register Page:**
 - `Login.js`
 - `Register.js`
 - `ForgotPassword.js`
- **Customer App:**
 - `HomePage.js` (Map view, ride request form, vehicle type selection)
 - `BookRidePage.js` (Detailed ride booking form)
 - `RideTrackingPage.js` (Real-time map with driver location, ride status)
 - `PaymentPage.js` (Payment method management, payment confirmation)
 - `RideHistoryPage.js` (List of past rides, ride details)
 - `ProfilePage.js` (Customer profile management)
 - `RatingPage.js` (Submit rating after ride completion)
- **Driver App:**
 - `DriverDashboard.js` (Current status, incoming ride requests, earnings summary)
 - `RideDetailsPage.js` (Details of an accepted ride, navigation)
 - `DriverProfilePage.js` (Driver profile, vehicle details, document upload)
 - `DriverRideHistoryPage.js` (List of past rides, earnings per ride)
- **Admin Panel:**
 - `AdminDashboard.js` (Overview of system metrics)
 - `UserManagementPage.js` (Manage customers, drivers, admins)
 - `DriverApprovalPage.js` (Approve/reject driver registrations)
 - `VehicleManagementPage.js` (Add/edit/delete vehicles)

- `RideMonitoringPage.js` (Monitor ongoing and past rides)
- `PaymentReportingPage.js` (View payment transactions, generate reports)
- `FareSettingsPage.js` (Configure pricing and surge rules)

5. Suggested Project Folder Structure

5.1. React Frontend

```
my-ride-hailing-app/
├── public/
│   └── index.html
├── src/
│   ├── assets/ (images, icons, etc.)
│   ├── components/ (reusable UI components)
│   │   ├── Auth/
│   │   │   ├── LoginForm.js
│   │   │   └── RegisterForm.js
│   │   ├── Common/
│   │   │   ├── Header.js
│   │   │   └── Footer.js
│   │   └── Maps/
│   │       └── MapComponent.js
│   ├── pages/ (main application views)
│   │   ├── Auth/
│   │   │   ├── LoginPage.js
│   │   │   └── RegisterPage.js
│   │   ├── Customer/
│   │   │   ├── HomePage.js
│   │   │   ├── BookRidePage.js
│   │   │   ├── RideTrackingPage.js
│   │   │   └── RideHistoryPage.js
│   │   ├── Driver/
│   │   │   ├── DriverDashboard.js
│   │   │   └── DriverRideHistoryPage.js
│   │   ├── Admin/
│   │   │   ├── AdminDashboard.js
│   │   │   └── UserManagementPage.js
│   │   └── NotFoundPage.js
│   ├── services/ (API calls, external integrations)
│   │   ├── auth.service.js
│   │   ├── user.service.js
│   │   └── ride.service.js
│   ├── contexts/ (React Context API for global state)
│   │   └── AuthContext.js
│   ├── hooks/ (custom React hooks)
│   ├── utils/ (utility functions)
│   ├── App.js (main application component)
│   └── index.js (entry point)
```

```
|   └─ styles/ (global styles, Bootstrap overrides)
├─ .env (environment variables)
├─ package.json
├─ README.md
```

5.2. Spring Boot Backend

```
ride-hailing-backend/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── ridehailing/
│   │   │   │   │   │   ├── RidehailingApplication.java (main application
class)
│   │   │   │   │   │   ├── config/ (Spring Security, JWT, etc.)
│   │   │   │   │   │   │   ├── SecurityConfig.java
│   │   │   │   │   │   │   └── JwtTokenProvider.java
│   │   │   │   │   │   ├── controller/ (REST API endpoints)
│   │   │   │   │   │   │   ├── AuthController.java
│   │   │   │   │   │   │   ├── UserController.java
│   │   │   │   │   │   │   ├── RideController.java
│   │   │   │   │   │   │   └── AdminController.java
│   │   │   │   │   │   ├── model/ (JPA entities, DTOs)
│   │   │   │   │   │   │   ├── User.java
│   │   │   │   │   │   │   ├── Customer.java
│   │   │   │   │   │   │   ├── Driver.java
│   │   │   │   │   │   │   ├── Vehicle.java
│   │   │   │   │   │   │   └── Ride.java
│   │   │   │   │   │   ├── repository/ (Spring Data JPA repositories)
│   │   │   │   │   │   │   ├── UserRepository.java
│   │   │   │   │   │   │   ├── RideRepository.java
│   │   │   │   │   │   │   └── DriverRepository.java
│   │   │   │   │   │   ├── service/ (Business logic)
│   │   │   │   │   │   │   ├── AuthService.java
│   │   │   │   │   │   │   ├── UserService.java
│   │   │   │   │   │   │   └── RideService.java
│   │   │   │   │   │   └── util/ (utility classes)
│   │   │   └── resources/
│   │   │       ├── application.properties (or application.yml) (configuration)
│   │   │       ├── static/ (frontend build files if served by backend)
│   │   │       └── templates/ (Thymeleaf, JSP, etc. if used for server-side
rendering)
│   └── test/
│       ├── java/
│       │   ├── com/
│       │   │   ├── example/
│       │   │   │   ├── ridehailing/
│       │   │   │   │   ├── RidehailingApplicationTests.java
```

```
├─ .gitignore
├─ pom.xml (Maven dependencies)
├─ README.md
```

6. Ride Booking Flow (Step-by-Step)

This section details the step-by-step process of a customer booking a ride, from login to payment.

1. User Login:

- Customer opens the React frontend application.
- Enters credentials (username/password) on the `LoginPage.js`.
- Frontend sends a `POST /api/auth/login` request to the Spring Boot backend.
- Backend authenticates the user, generates a JWT token, and sends it back to the frontend.
- Frontend stores the JWT token (e.g., in local storage) and redirects the user to the `HomePage.js`.

2. Selecting Pickup & Drop Location:

- On the `HomePage.js`, the customer sees a map and input fields for pickup and drop-off locations.
- Customer enters pickup and drop-off addresses (or selects on the map).
- Frontend uses a mapping service API (e.g., Google Maps API) to convert addresses to geographical coordinates (latitude, longitude).
- Frontend sends a `GET /api/fare/estimate` request to the backend with pickup/drop-off coordinates and selected vehicle type.
- Backend calculates the estimated fare using the Pricing & Fare Calculation Module and returns it to the frontend.
- Customer reviews the estimated fare and confirms the ride request.

3. Matching Driver:

- Frontend sends a `POST /api/rides/request` to the backend with ride details.
- Backend receives the ride request and initiates the Ride Matching Module.
- The Ride Matching Module queries the database for `drivers` with `status = 'AVAILABLE'` and `current_location` within a certain radius of the pickup location.
- The backend sends a notification (via Notification Module) to the nearest available drivers about the new ride request.
- Drivers receive the notification on their `DriverDashboard.js` and can view ride details (pickup, drop-off, estimated fare).
- A driver accepts the ride by sending a `POST /api/drivers/{driverId}/accept-ride` request.
- Upon acceptance, the backend updates the `rides` table (`driver_id`, `status = 'ACCEPTED'`) and the driver's status (`status = 'ON_RIDE'`).
- The customer receives a notification that a driver has accepted their ride, along with driver and vehicle details.

4. Ride Start:

- The driver navigates to the pickup location.
- Once the driver arrives and the customer boards, the driver initiates the ride start on their `DriverDashboard.js`.
- Frontend sends a `PUT /api/rides/{id}/start` request to the backend.
- Backend updates the `rides` table (`status = 'IN_PROGRESS'`, `start_time`).
- The `RideTrackingPage.js` on the customer's app starts displaying the driver's real-time location (via `PUT /api/drivers/{id}/location` updates from the driver app and `GET /api/rides/{id}/track` from the customer app).

5. Ride Completion:

- Upon reaching the drop-off location, the driver ends the ride on their `DriverDashboard.js`.

- Frontend sends a `PUT /api/rides/{id}/complete` request to the backend.
- Backend calculates the final fare (considering actual distance, time, and any surge pricing) and updates the `rides` table (`status = 'COMPLETED'` , `end_time` , `actual_fare`).
- The driver's status is updated back to `AVAILABLE` .
- The customer receives a notification about the ride completion and the final fare.

6. Payment:

- The customer is prompted to make a payment on the `PaymentPage.js` .
- Frontend sends a `POST /api/payments/process` request to the backend with the ride ID and selected payment method.
- Backend interacts with a third-party payment gateway to process the transaction.
- Upon successful payment, the `payments` table is updated (`status = 'COMPLETED'`).
- The customer receives a payment confirmation.
- The customer is then prompted to rate the driver on the `RatingPage.js` .

7. Future Features

Here are some suggested future features to enhance the system:

- **Real-time Tracking Enhancements:** Implement more granular real-time tracking with estimated time of arrival (ETA) predictions, route optimization, and live traffic updates using advanced mapping APIs and algorithms.
- **Surge Pricing Automation:** Develop a more sophisticated algorithm for dynamic surge pricing based on demand-supply ratios, historical data, and special events. This would be managed through the Admin Dashboard.
- **Promotions and Discounts:** Implement a module for creating and managing promotional codes and discounts, allowing customers to apply them during ride booking. This would involve new database tables for `promotions` and `user_promotions` .

- **Driver Earnings Dashboard:** Provide drivers with a detailed dashboard showing their daily, weekly, and monthly earnings, ride statistics, and performance metrics. This would leverage data from the `rides` and `payments` tables.
- **Ride Cancellation System with Penalties:** Implement a robust ride cancellation policy, including cancellation fees for customers or drivers under certain conditions. This would involve tracking cancellation reasons and applying penalties, which would impact the Payment Module.
- **Scheduled Rides:** Allow customers to schedule rides in advance, which would require a scheduling mechanism within the Ride Booking and Ride Matching Modules.
- **Multiple Stops:** Enable customers to add multiple stops during a single ride, requiring adjustments to fare calculation and route planning.
- **Emergency/SOS Button:** Integrate an emergency button for both customers and drivers to alert authorities or emergency contacts in case of an incident.
- **In-App Chat:** Implement a chat feature between customers and drivers for communication before and during a ride.
- **Wallet System:** Introduce an in-app wallet where users can add money and use it for payments, offering a seamless payment experience.

8. Conclusion

This document provides a foundational system design for a Car Rental / Ride-Hailing application. By following this structured approach, you can systematically build a robust, scalable, and feature-rich platform. Remember to prioritize security, performance, and user experience throughout the development process. Happy coding!